

240902 세팅 정리

- root-context.xml

→ 주로 Spring framework 사용 시 애플리케이션의 전역 설정(예 : 서비스, 리포지토리, 데이터베이스 연결 등) 을 정의한다.

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- 'xmlns' = 각 네임스페이스 선언으로, XML문서가 어떤 XML 스키마를
      예를 들어, 'http://www.springframework.org/schema/beans'

      xsi:schemaLocation : XML 문서에서 사용되는 각 네임스페이스에
      이 정보는 XML 문서의 유효성을 검증하는데 사용됨.
-->
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:mybatis-spring="http://mybatis.org/schema/mybatis-spring"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context
                           http://mybatis.org/schema/mybatis-spring
                           http://mybatis.org/schema/mybatis-spring"

    <!-- Root Context: defines shared resources visible to all descendant contexts -->

    <!-- Bean의 이름을 'dataSource'로 지정. 다른 Bean에서 이 DataSource을 참조할 수 있다 -->
    <!-- org.apache.commons.dbcp2.BasicDataSource 이 Bean이 정의된다 -->
    <!-- 여기서는 Apache Commons DBCP(데이터베이스 커넥션 풀) 라이브러리를 사용한다 -->
    <!-- destroy-method="close" 애플리케이션이 종료될 때 이 Bean의 close 메서드는 DataSource를 종료하여 연결 풀을 정리한다. -->
    <bean id="dataSource" class="org.apache.commons.dbcp2.BasicDataSource"
        <!-- driverClassName : 데이터베이스 연결에 사용할 JDBC 드라이버 클래스명 -->
```

```

        <property name="driverClassName" value="oracle.jdbc
        <!-- url : 데이터베이스에 연결할 URL. 마찬가지로 Oracle D
        <property name="url" value="jdbc:oracle:thin:@172.
        <!-- username : 데이터베이스에 연결할 때 사용할 사용자 이름
        <property name="username" value="team8"/>
        <!-- password : 데이터베이스에 연결할 때 사용할 비밀번호 -
        <property name="password" value="1234"/>
    </bean>

    <!-- id="sqlSessionFactory : MyBatis와 Spring이 연동되어
        class="org.mybatis.spring.SqlSessionFactoryBean :
    -->
    <bean id="sqlSessionFactory" class="org.mybatis.spring
        <!-- dataSource Bean을 참조. 이를 통해서 MyBatis가 데이
        <property name="dataSource" ref="dataSource"/>
        <!-- configLocation : MyBatis의 설정 파일 경로를 지정한
        <property name="configLocation" value="classpath:m
        <!-- mapperLocations : MyBatis 매퍼 XML 파일들의 위치를
        classpath:mybatis/mappers/*.xml은 클래스패스 내의
        <property name="mapperLocations" value="classpath:
        <!-- typeAliasesPackage : MyBatis에서 사용할 타입 별칭
        이 설정을 통해 도메인 객체의 패키지를 지정하여 XML 매퍼
        <property name="typeAliasesPackage" value="kr.ac.n
    </bean>

    <!-- id="sqlSessionTemplate": SqlSessionTemplate은 MyBa
        SQL 실행, 매핑, 트랜잭션 관리 등을 처리 -->
    <bean id="sqlSessionTemplate" class="org.mybatis.spring
        <!-- constructor-arg: 이 SqlSessionTemplate Bean을
        위에서 정의한 sqlSessionSessionFactory Bean을 참조. -->
        <constructor-arg name="sqlSessionFactory" ref="sql
    </bean>

    <!-- mybatis-spring:scan: 지정된 패키지에서 MyBatis DAO(데
    <!-- base-package: DAO 인터페이스가 위치한 기본 패키지를 지정.
        **는 모든 하위 패키지를 포함한다는 의미
    -->
    <mybatis-spring:scan base-package="kr.ac.nollrae.**.da

</beans>

```

요약 : 이 root-context.xml 파일은 Spring 프레임워크와 MyBatis를 사용하여 데이터베이스와의 연동을 설정한다.

dataSource를 정의하여 데이터베이스 연결을 관리하고,

sqlSessionFactory와 sqlSessionTemplate을 통해 MyBatis가 데이터베이스와 상호 작용할 수 있도록 구성한다.

마지막으로 mybatis-spring:scan 태그를 통해 지정된 패키지의 DAO들을 스캔하고 자동으로 Bean으로 등록하여 쉽게 데이터베이스에 접근할 수 있게 한다.

- web.xml

→ Java 웹 애플리케이션의 기본 설정 파일. 서블릿, 필터, 리스터 등의 매핑을 정의한다. 예를 들어 'DispatcherServlet'이나 보안 필터 등을 설정할 때 사용된다. 서블릿 컨테이너가 애플리케이션을 시작할 때 읽는 첫 번째 파일이다.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5" xmlns="http://java.sun.com/xml/ns/j2ee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">

  <!-- The definition of the Root Spring Container shared by all Servlets and Filters -->
  <!-- - context-param: 모든 서블릿과 필터에서 공유할 수 있는 전역 설정 -->
  <!-- - contextConfigLocation: Spring 프레임워크에서 사용하는 Spring 컨텍스트의 위치 -->
  <!-- 여기서 root-context.xml 파일이 지정되어 있으며, 이 파일은 모든 서블릿과 필터에 공유됨. -->
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/spring/root-context.xml</param-value>
  </context-param>

  <!-- Creates the Spring Container shared by all Servlets and Filters -->
  <!-- - listener: 특정 이벤트가 발생할 때 실행될 코드를 정의 -->
  <!-- - ContextLoaderListener: Spring의 ContextLoaderListener 클래스를 사용하여 -->
  <!-- root-context.xml 파일을 읽고, 애플리케이션의 루트 Spring 컨텍스트를 생성 -->
  <!-- 이 컨텍스트는 모든 서블릿과 필터에 공유됨. -->
  <listener>
```

```

        <listener-class>org.springframework.web.context.Co
</listener>

<!-- Processes application requests -->
<!-- - servlet: 웹 애플리케이션에서 요청을 처리하는 서블릿을 정의
- DispatcherServlet: Spring MVC에서 사용하는 주요 서블릿
- init-param: DispatcherServlet이 사용할 추가적인 Spring
여기서는 servlet-context.xml과 view-controller.xml
- load-on-startup: 이 서블릿이 애플리케이션 시작 시 초기화
값이 1이므로 애플리케이션 시작 시점에 가장 먼저 초기화 된다
-->
<servlet>
    <servlet-name>appServlet</servlet-name>
    <servlet-class>org.springframework.web.servlet.Dis
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/spring/appServlet/servle
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>

<!-- - servlet-mapping: 특정 URL 패턴이 어떤 서블릿에 매핑될지
- /: 이 매핑은 모든 요청을 appServlet으로 전달하도록 설정
-->
<servlet-mapping>
    <servlet-name>appServlet</servlet-name>
    <url-pattern>/</url-pattern>
</servlet-mapping>

<!-- 한글설정 -->
<!-- - filter: 웹 애플리케이션의 요청 및 응답을 필터링하는 로직
- CharacterEncodingFilter: 모든 요청의 인코딩을 UTF-8
이 필터를 통해 애플리케이션에서 처리되는 모든 문자 데이터가
- filter-mapping: 이 필터가 적용될 URL 패턴을 정의. '/'
-->
<filter>
    <filter-name>encodingFilter</filter-name>
    <filter-class>org.springframework.web.filter.Chara

```

```

        <init-param>
            <param-name>encoding</param-name>
            <param-value>UTF-8</param-value>
        </init-param>
        <init-param>
            <param-name>forceEncoding</param-name>
            <param-value>true</param-value>
        </init-param>
    </filter>
    <filter-mapping>
        <filter-name>encodingFilter</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>
    <!-- 한글설정 END -->

</web-app>

```

요약 : `web.xml` 파일은 Spring MVC 기반 웹 애플리케이션의 초기 설정을 정의한다. 애플리케이션이 시작되면 Spring 컨텍스트가 초기화되고, 모든 요청은 `DispatcherServlet`에 의해 처리된다. 또한, 모든 요청과 응답의 인코딩을 UTF-8로 강제하여 한글이나 기타 유니코드 문자가 올바르게 처리되도록 한다.

전체적인 흐름은 애플리케이션 초기화 → Spring 컨텍스트 로드 → DispatcherServlet 설정 및 매핑 → CharacterEncodingFilter 적용 순

- pom.xml
→ Maven 프로젝트의 기본 구성 파일. 프로젝트의 의존성, 빌드 설정, 플러그인 등을 정의한다. Maven을 통해 프로젝트를 빌드하고 관리할 때 필수적인 파일.

요약 :

- servlet-context.xml

→ Spring MVC에서 주로 사용되며, 서블릿 관련 설정을 정의한다. 컨트롤러, 뷰 리졸버, 메시지 컨버터 등의 설정이 포함된다. 'web.xml'에서 'DispatcherServlet'을 매핑했다면, 그 서블릿에 대한 설정이 이 파일에 정의된다.

요약:

- view-controller.xml

→ 주로 뷰를 매핑하기 위해 사용한다. 예를 들어 특정 URL에 대한 간단한 뷰 리졸루션을 정의할 때 사용한다. Spring MVC에서 사용되는 경우가 많다.

요약 :

- mybatis-config.xml

→ MyBatis 프레임워크의 설정 파일. MyBatis는 SQL 매핑 프레임워크로, 이 파일에서 데이터베이스 연결 정보, 매퍼 설정, 트랜잭션 관리 등을 설정할 수 있다.

요약 :

- 순서는 일반적으로 전반적인 설정에서 구체적인 설정으로 이동하는 흐름이다.

`web.xml` 이 서블릿 컨테이너의 초기 설정을 담당하고,

그 후에 `root-context.xml` 과 `servlet-context.xml` 이 애플리케이션의 전반적 및 서블릿 관련 설정을 다루며,

`mybatis-config.xml` 과 같은 특정 기술의 설정 파일이 그 뒤를 이을 수 있다.

- 아래 두가지는 순서를 굳이 매기지 않는다.
 - pom.xml은 프로젝트 전체의 빌드 및 의존성 관리를 담당한다. 따라서 순서를 따지기 보다는 전반적인 프로젝트의 설정과 관련이 있다.
 - view-xontroller.xml은 Spring MVC설정의 일부로, 보통 컨트롤러나 뷰 매핑을 위한 간단한 설정을 포함하는 파일이다.
-